

DOSSIER D'ANALYSE

Projet N°7

Système HIPS sous Linux

Équipe Groupe 2

- KAMDOUM TEUFENG Josette
- HARVIS LANDRY FOTSEU FONKWA
- NONGA Bikim



Phase 2 du projet

Analyse & Modélisation

Livrable

Dossier d'analyse

Méthode

UML 2.x

Encadrant

NGUIMBUS Emmanuel

Année académique 2024-2025

Plan du dossier

Structure complète de l'analyse

INTRODUCTION

I – PRÉSENTATION

A. Contexte

B. Définition du problème

II – BESOIN DU SYSTÈME

A. Besoins fonctionnels

B. Besoins non fonctionnels

III – CAS D'UTILISATION DU SYSTÈME

A. Acteurs internes

B. Acteurs externes

C. Cas d'utilisations proprement dit

D. Diagramme de cas d'utilisation

INTRODUCTION

Présentation du dossier et du contexte du projet

A. Contexte — Le défi de la cybersécurité sous Linux

Environnement actuel et enjeux de la sécurité hôte

Linux est le système d'exploitation dominant dans les environnements serveurs mondiaux. Sa robustesse et sa flexibilité en font le pilier des infrastructures cloud, des serveurs web et des systèmes critiques. Cependant, cette omniprésence en fait également une cible privilégiée des attaquants.

Menaces croissantes

Les attaques ciblant les systèmes Linux ont augmenté de façon continue ces dernières années : botnets, ransomwares, mineurs de cryptomonnaies, backdoors.

Journaux inexploités

Les serveurs Linux génèrent des millions de lignes de logs par jour (syslog, auditd). Sans analyse automatisée, ces données restent inutilisées et les attaques passent inaperçues.

Réponse manuelle insuffisante

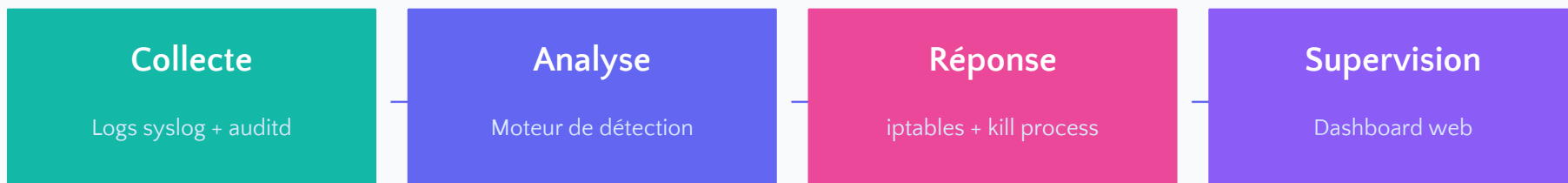
Lorsqu'une anomalie est détectée manuellement, il est souvent trop tard. Une réponse automatique en temps réel est indispensable pour limiter les dégâts.

C'est dans ce contexte qu'est né ce projet : concevoir un système HIPS (Host Intrusion Prevention System) opérationnel sous Linux, capable de surveiller automatiquement les activités système, détecter les comportements malveillants en temps réel et déclencher des contre-mesures automatiques sans intervention humaine.

A. Contexte — Qu'est-ce qu'un système HIPS ?

Host Intrusion Prevention System : définition et architecture

Un système HIPS (Host Intrusion Prevention System) est un logiciel de sécurité installé directement sur un hôte informatique. Contrairement aux IDS (Intrusion Detection Systems) qui se contentent de détecter, un HIPS détecte ET agit automatiquement pour bloquer ou contenir la menace.



Surveillance continue	24h/24, 7j/7 sur les journaux système en temps réel
Détection comportementale	Analyse des patterns suspects, pas uniquement des signatures connues
Réponse automatique	Blocage IP, arrêt de processus sans intervention humaine
Interface web	Tableau de bord accessible depuis n'importe quel navigateur
Simulation intégrée	Module de test pour valider l'ensemble de la chaîne de sécurité

La question fondamentale à laquelle ce projet répond

« Comment détecter en temps réel les intrusions et comportements malveillants sur un système Linux hôte, et y répondre automatiquement avant qu'ils ne causent des dommages irréversibles sur l'infrastructure ? »

Cette problématique se décompose en quatre sous-problèmes interdépendants :

1

Absence de supervision centralisée

Les logs syslog et auditd sont dispersés dans plusieurs fichiers. Sans centralisation ni corrélation, il est impossible de détecter une attaque dans un flux de millions d'entrées.

2

Détection partielle et réactive

Les outils existants (fail2ban, antivirus) ne couvrent qu'une partie des vecteurs d'attaque. Aucun outil ne surveille simultanément les brute force, les accès fichiers sensibles et l'exécution de malwares.

3

Pas de réponse automatique cohérente

L'intervention est manuelle et tardive. Un attaquant dispose de plusieurs minutes à plusieurs heures pour agir entre la détection et le blocage. Ce délai est inacceptable.

4

Impossibilité de valider le système

Sans module de simulation d'attaques intégré, il est impossible de tester que les mécanismes de détection et de réponse fonctionnent correctement avant un incident réel.

B. Définition du problème – Analyse des causes

Pourquoi ces problèmes existent et comment les résoudre

L'analyse des causes profondes permet de mieux comprendre l'origine des problèmes et de concevoir des solutions adaptées. Le tableau ci-dessous présente pour chaque problème sa cause racine et la solution retenue dans notre système HIPS.

Problème	Cause racine	Solution HIPS
Logs dispersés et non analysés	Absence d'outil de centralisation et de corrélation des journaux	Module de collecte : lecture temps réel syslog + auditd, stockage BDD
Détection partielle	Outils mono-vecteur (fail2ban = SSH uniquement)	Moteur multi-règles : brute force, fichiers sensibles, processus suspects
Réponse manuelle tardive	Pas de mécanisme de déclenchement automatique de contre-mesures	Module de réponse : iptables automatique + kill process en < 30 secondes
Système non testable	Aucun environnement de simulation d'attaques disponible	Module de simulation : 3 scénarios d'attaque reproductibles et mesurables

Impact attendu de la solution :

Réduction du délai de détection à < 30 secondes · Couverture de 3 vecteurs d'attaque simultanément · Supervision centralisée via dashboard web
· Validation possible par simulation

Section II

BESOIN DU SYSTÈME

Besoins fonctionnels et non fonctionnels du système HIPS

Ce que le système doit faire – Fonctions principales

Les besoins fonctionnels décrivent l'ensemble des fonctions que le système HIPS doit remplir pour répondre aux objectifs du projet. Ils sont organisés par module.

BF01 Collecte de logs

Collecter les journaux syslog en temps réel

Le système doit lire en continu le fichier `/var/log/syslog` et toutes ses rotations. Chaque nouvelle entrée doit être capturée dès son écriture, parsée et stockée en base de données dans un délai inférieur à 10 secondes.

BF02 Collecte de logs

Collecter les journaux auditd en temps réel

Le système doit lire le fichier `/var/log/audit/audit.log` produit par le démon `auditd`. Les règles `auditd` doivent être configurées pour surveiller les accès aux fichiers sensibles, les appels système critiques et les exécutions de processus.

BF03 Détection

Détecter les attaques par force brute SSH

Lorsque 5 échecs d'authentification SSH provenant de la même IP sont détectés en moins de 60 secondes, le système doit générer une alerte de niveau CRITIQUE et déclencher le blocage automatique de l'IP source.

BF04 Détection

Détecter les accès non autorisés aux fichiers sensibles

Tout accès (lecture ou écriture) sur les fichiers `/etc/shadow`, `/etc/passwd`, `/etc/sudoers`, les répertoires SSH (`/root/.ssh`, `/home/*/.ssh`) et les certificats SSL par un utilisateur ou processus non autorisé doit générer une alerte.

Fonctions de réponse, supervision et simulation

BF05 – Détection

Détecter l'exécution de processus suspects et malwares

Tout processus exécuté depuis /tmp, /dev/shm, dont le nom ne figure pas dans la liste blanche, lancé avec des droits root par un utilisateur non admin, ou consommant > 80% CPU pendant > 30s, doit déclencher une alerte de niveau ÉLEVÉ.

BF06 – Réponse

Bloquer automatiquement les IP malveillantes

Lors d'une détection de type force brute ou intrusion réseau, le système doit automatiquement ajouter une règle DROP dans iptables pour l'IP source, dans les deux sens, et enregistrer cette action en base de données.

BF07 – Réponse

Arrêter automatiquement les processus malveillants

Lors d'une détection de processus malveillant, le système doit envoyer un signal SIGKILL au processus identifié, enregistrer le PID, le nom et l'utilisateur dans les logs d'actions, et afficher l'événement sur le dashboard.

BF08 – Dashboard

Afficher les alertes en temps réel sur l'interface web

Le dashboard doit afficher les nouvelles alertes en temps réel (< 5 secondes après génération) avec : niveau de sévérité, type d'attaque, IP/processus concerné, horodatage et statut de la contre-mesure déclenchée.

BF09 – Dashboard

Consulter et filtrer l'historique des événements

L'administrateur doit pouvoir consulter l'historique complet des événements et alertes, avec filtres par date, type, sévérité. Une fonction d'export CSV doit être disponible. Les IP bloquées doivent être listées avec option de déblocage.

BF10 – Simulation

Simuler des attaques pour valider le système

Le système doit inclure un module permettant d'exécuter 3 scénarios d'attaque : brute force SSH, accès /etc/shadow, exécution malware depuis /tmp. Les

B. Besoins non fonctionnels

Contraintes de qualité, performance et sécurité

Les besoins non fonctionnels définissent les propriétés et contraintes qualitatives que le système HIPS doit respecter, indépendamment des fonctionnalités métier.

Performance

BNF01	Délai de détection	Toute menace doit être détectée en moins de 30 secondes après son apparition dans les logs
BNF02	Délai d'affichage	Les alertes doivent apparaître sur le dashboard en moins de 5 secondes après génération
BNF03	Throughput	Le moteur de détection doit traiter au minimum 1 000 événements logs par minute sans perte

Disponibilité

BNF04	Uptime	L'agent de surveillance doit fonctionner 24h/24, 7j/7 avec redémarrage automatique en cas de crash
BNF05	Résilience	En cas de panne de la base de données, la collecte de logs doit continuer avec mise en file d'attente

Sécurité

BNF06	Authentification	L'accès au dashboard web doit être protégé par login/mot de passe (hashé avec bcrypt)
BNF07	Intégrité des logs	Les logs d'actions du HIPS doivent être protégés en écriture pour éviter toute falsification
BNF08	Confidentialité	Les communications avec le dashboard peuvent être chiffrées via HTTPS

Section III

CAS D'UTILISATION DU SYSTÈME

Acteurs, cas d'utilisation et diagramme UML

Participants internes au système HIPS

Les acteurs internes sont les entités qui font partie intégrante du système ou qui opèrent directement en son sein. Ils interagissent entre eux pour assurer le fonctionnement du HIPS.

Administrateur système

L'administrateur est le seul utilisateur humain du système. Il accède au dashboard web pour consulter les alertes, visualiser l'historique, gérer les IP bloquées et lancer des simulations d'attaques. Il configure également les règles de détection (seuils, listes blanches).

Actions

- Consulter le dashboard de supervision
- Visualiser et filtrer les alertes
- Consulter l'historique des événements
- Débloquer une IP manuellement
- Lancer une simulation d'attaque
- S'authentifier sur le dashboard

Agent HIPS (Système)

L'agent HIPS est le processus principal qui tourne en arrière-plan sur le serveur Linux. Il collecte les logs, les analyse via le moteur de détection, déclenche les réponses automatiques et alimente la base de données. Il fonctionne de manière entièrement autonome.

Actions

- Collecter les logs syslog et auditd
- Analyser les événements en temps réel
- Déclencher les contre-mesures automatiques
- Alimenter la base de données
- Générer les alertes

Moteur de détection

Sous-composant de l'Agent HIPS, le moteur de détection applique les règles d'analyse sur les événements normalisés. Il prend les décisions de classification (normal / suspect / malveillant) et notifie l'agent pour le déclenchement des réponses.

Actions

- Évaluer les règles de détection
- Classifier les événements
- Calculer les scores de menace
- Déclencher les alertes

Entités extérieures interagissant avec le système

Les acteurs externes sont des entités qui existent en dehors du système HIPS mais qui interagissent avec lui, soit en tant que source de menaces, soit en tant que composants système sur lesquels le HIPS s'appuie.

Noyau Linux / VFS

Le noyau Linux, via la couche VFS (Virtual File System), génère les événements d'audit captés par auditd. Il est responsable de la création, lecture et suppression des fichiers, et notifie auditd de chaque opération selon les règles configurées. Le HIPS s'appuie sur lui comme source primaire d'information.

- Génère les événements d'accès aux fichiers
- Notifie auditd des appels système
- Enregistre les activités réseau et processus

Service SSH (sshd)

Le démon SSH est la cible principale des attaques de type brute force. Il enregistre chaque tentative de connexion (réussie ou échouée) dans les journaux syslog (/var/log/auth.log). Le HIPS surveille ces logs pour détecter les patterns d'attaque répétés.

- Journalise les tentatives d'authentification
- Enregistre les connexions SSH réussies
- Source principale des alertes brute force

Attaquant simulé

Dans le cadre du module de simulation, l'attaquant simulé est représenté par la machine attaquante (Kali Linux) qui exécute les scénarios d'attaque. En production, il représente tout acteur malveillant tentant de compromettre le système depuis le réseau ou depuis un compte compromis.

- Lance des attaques brute force SSH

iptables / nftables

iptables est le pare-feu noyau Linux utilisé par le module de réponse automatique pour bloquer les IP malveillantes. Le HIPS interagit avec iptables en ajoutant des règles DROP dynamiquement. C'est un acteur externe car il existe indépendamment du HIPS.

- Reçoit les règles de blocage IP

Intitulé

Surveiller les logs système

Objectif

Assurer la collecte continue et automatique des journaux syslog et auditd, les normaliser et les stocker en base de données pour analyse.

Résumé

L'Agent HIPS lit en temps réel les fichiers de journaux système. Chaque nouvelle entrée est parsée, normalisée dans un format commun et stockée dans la base de données

Acteurs

Internes : Agent HIPS, Moteur de détection

Externes : *Noyau Linux / VFS, Service SSH*

Préconditions

- Le système Linux est démarré
- Le service auditd est actif
- Les droits de lecture sur les fichiers logs sont accordés

Postconditions

- Les nouveaux événements sont enregistrés en BDD
- Le moteur de détection peut accéder aux événements
- Un horodatage précis est associé à chaque entrée

Scénario nominal

- 1 L'Agent HIPS ouvre les fichiers `/var/log/syslog` et `/var/log/audit/audit.log`
- 2 Une nouvelle entrée est écrite dans le fichier log
- 3 L'Agent détecte la modification
- 4 L'entrée est parsée et normalisée au format JSON
- 5 L'événement normalisé est inséré en base de données
- 6 Le Moteur de détection est notifié du nouvel événement

Scénario alternatif / Exceptions

- ⚠ Fichier log inaccessible → alerte système, tentative après 30s
- ⚠ Base de données indisponible → mise en file d'attente locale (buffer)
- ⚠ Entrée malformée → ignorée et comptabilisée dans les métriques d'erreur

Cas d'utilisation 2 : Détecter une attaque brute force SSH

III — Cas d'utilisation

Intitulé

Détecter une attaque brute force SSH

Objectif

Identifier automatiquement les tentatives répétées de connexion SSH échouées depuis la même adresse IP et déclencher une alerte de niveau CRITIQUE.

Résumé

Lors d'une attaque brute force SSH, l'attaquant génère de nombreux échecs d'authentification. Le moteur de détection analyse ces patterns et déclenche une alerte lorsque le seuil configurable est atteint.

Acteurs

Internes : Moteur de détection, Agent HIPS

Externes : *Attaquant simulé, Service SSH (sshd)*

Préconditions

- La collecte de logs est active
- Le service sshd est en cours d'exécution
- La règle brute force est configurée (seuil = 5 échecs / 60s)

Postconditions

- Une alerte CRITIQUE est enregistrée en base de données
- L'administrateur est notifié via le dashboard
- Le module de réponse est déclenché automatiquement

Scénario nominal

- 1 L'attaquant envoie une tentative de connexion SSH avec mauvais mot de passe
- 2 sshd écrit un échec d'authentification dans /var/log/auth.log
- 3 L'Agent HIPS détecte la nouvelle entrée
- 4 Le Moteur de détection extrait l'IP source et l'horodatage

Scénario nominal (suite)

- 5 Le compteur d'échecs pour cette IP est incrémenté
- 6 Au 3e échec en moins de 60s, le seuil est atteint
- 7 Une alerte CRITIQUE est créée et la réponse automatique déclenchée

Scénario alternatif / Exceptions

⚠ Moins de 5 tentatives → alerte WARNING uniquement, pas de blocage

⚠ IP déjà bloquée → alerte ignorée, IP déjà dans iptables

⚠ Attaque distribuée (plusieurs IP) → alertes distinctes par IP source

Cas d'utilisation 3 : Détecter un accès non autorisé à un fichier sensible

III — Cas d'utilisation

Intitulé

Détecter un accès non autorisé à un fichier sensible

Objectif

Détecter toute tentative d'accès (lecture ou écriture) à des fichiers critiques du système par un utilisateur ou processus non autorisé.

Résumé

Via les règles auditd, chaque accès aux fichiers sensibles de la liste de surveillance est capturé. Le moteur de détection vérifie si l'utilisateur ou le processus est autorisé et génère une alerte si ce n'est pas le cas.

Acteurs

Internes : Moteur de détection, Agent HIPS

Externes : *Noyau Linux / VFS, Attaquant simulé*

Préconditions

- Le service auditd est actif avec les règles de surveillance des fichiers
- La liste des fichiers sensibles est configurée (/etc/shadow, /etc/passwd, etc.)
- La liste blanche des processus et utilisateurs autorisés est maintenue

Postconditions

- Une alerte ÉLEVÉ est générée et enregistrée en base
- L'événement est visible sur le dashboard avec le fichier ciblé et l'utilisateur
- Le fichier n'a pas été modifié (en cas d'écriture bloquée)

Scénario nominal

- 1 Un utilisateur non autorisé tente de lire un fichier sensible
- 2 Le noyau Linux traite l'appel système open()
- 3 auditd capture l'événement OPEN_READ
- 4 L'Agent HIPS détecte la nouvelle entrée auditd

Cas d'utilisation 3 : Détecter un accès non autorisé à un fichier sensible (suite)

Scénario nominal (suite)

- 5 Le Moteur vérifie si l'utilisateur figure dans la liste blanche
- 6 L'utilisateur n'est pas autorisé → alerte de niveau ÉLEVÉ générée
- 7 Le dashboard affiche l'alerte avec le détail de l'accès

Scénario alternatif / Exceptions

- ⚠ Utilisateur root (autorisé) → accès enregistré mais pas d'alerte
- ⚠ Processus système légitimes (cron, ldap) → ignorés via liste blanche
- ⚠ Accès depuis un processus inconnu → alerte CRITIQUE avec investigation recommandée

Cas d'utilisation 4 : Détecter l'exécution d'un processus suspect

III — Cas d'utilisation

Intitulé Détecter l'exécution d'un processus suspect

Objectif

Identifier l'exécution de binaires ou scripts depuis des répertoires non autorisés, ou de processus dont le comportement est anormal (consommation CPU excessive, connexions réseau suspectes).

Résumé
auditd surveille les appels système EXECVE. Le moteur vérifie chaque nouveau processus lancé contre une liste blanche et des règles comportementales. Tout processus non conforme déclenche une alerte.

Acteurs

Internes : Moteur de détection, Agent HIPS

Externes : Noyau Linux / VFS, Attaquant simulé

Préconditions

- auditd surveille les événements EXECVE
- La liste blanche des processus légitimes est à jour
- Le module de surveillance CPU est actif

Postconditions

- Une alerte ÉLEVÉ ou CRITIQUE est générée selon le niveau de menace
- Le processus suspect est identifié par son PID, nom et utilisateur
- Le module de réponse automatique est notifié pour kill process

Scénario nominal

- 1 Un attaquant dépose un script malveillant
- 2 Le script est exécuté (chmod +x puis ./script.sh)
- 3 auditd capture l'événement EXECVE avec le chemin d'exécution
- 4 L'Agent HIPS détecte la nouvelle entrée

Cas d'utilisation 4 : Détecter l'exécution d'un processus suspect (suite)

Scénario nominal (suite)

- 5 Le Moteur vérifie répertoire suspect
- 6 Une alerte CRITIQUE est générée
- 7 Le module de réponse envoie SIGKILL au processus

Scénario alternatif / Exceptions

- ⚠ Processus dans la liste blanche → aucune alerte
- ⚠ Faux positif (script de maintenance) → alerte marquée à réviser
- ⚠ Processus déjà terminé avant kill → alerte enregistrée, action ignorée

Cas d'utilisation 5 : Bloquer automatiquement une adresse IP malveillante

III — Cas d'utilisation

Intitulé
Bloquer automatiquement une adresse IP malveillante

Objectif

Ajouter automatiquement une règle de filtrage dans iptables pour bloquer toute communication avec une adresse IP identifiée comme malveillante, sans intervention humaine.

Résumé
Suite à la détection d'une attaque réseau, le module de réponse automatique construit et exécute la commande iptables appropriée pour bloquer l'IP source dans les deux sens (entrée et sortie).

Acteurs

Internes : Agent HIPS, Moteur de détection

Externes : *iptables / nftables, Attaquant simulé*

Préconditions

- Une alerte de niveau CRITIQUE a été générée avec une IP source identifiée
- L'Agent HIPS dispose des droits sudo pour modifier les règles iptables
- L'IP n'est pas déjà dans la liste de blocage

Postconditions

- Une règle DROP est ajoutée pour l'IP source dans iptables (INPUT et OUTPUT)
- L'action est enregistrée en base de données avec horodatage
- L'alerte sur le dashboard indique 'IP bloquée'
- L'attaquant ne peut plus communiquer avec le serveur

Scénario nominal

- 1 Le Moteur de détection génère une alerte CRITIQUE avec IP source
- 2 L'Agent HIPS reçoit la notification de blocage nécessaire
- 3 L'Agent vérifie que l'IP n'est pas déjà bloquée
- 4 L'Agent construit la commande : iptables -A INPUT -s <IP> -j DROP

Cas d'utilisation 5 : Bloquer automatiquement une adresse IP malveillante (suite)

Scénario nominal (suite)

- 5 L'Agent exécute la commande via subprocess avec sudo
- 6 La commande est également appliquée en OUTPUT
- 7 L'action est enregistrée en base et le dashboard mis à jour

Scénario alternatif / Exceptions

- ⚠ IP déjà bloquée → action ignorée, doublon évité
- ⚠ Erreur iptables (droits insuffisants) → alerte système, escalade à l'admin
- ⚠ IP dans la liste blanche → blocage annulé, alerte WARNING générée

Cas d'utilisation 6 : Arrêter automatiquement un processus malveillant

III — Cas d'utilisation

Intitulé Arrêter automatiquement un processus malveillant

Objectif

Terminer immédiatement un processus identifié comme malveillant en lui envoyant un signal SIGKILL, et journaliser l'action dans les logs du HIPS.

Résumé

Lorsque le moteur de détection identifie un processus malveillant par son PID, le module de réponse envoie SIGKILL pour l'arrêter immédiatement sans lui laisser la possibilité d'effectuer des actions de nettoyage.

Acteurs

Internes : Agent HIPS, Moteur de détection

Externes : Noyau Linux / VFS

Préconditions

- Un processus malveillant a été identifié avec son PID
- L'Agent HIPS dispose des droits nécessaires pour envoyer des signaux
- Le processus est encore en cours d'exécution

Postconditions

- Le processus malveillant est terminé (code retour 0 de kill -9)
- L'action est enregistrée en base : PID, nom, utilisateur, raison, horodatage
- Une entrée est ajoutée dans /var/log/hips/actions.log
- Le dashboard affiche l'événement avec le statut 'Processus arrêté'

Scénario nominal

- 1 Le Moteur détecte un processus suspect avec son PID
- 2 L'Agent reçoit l'instruction d'arrêt avec le PID cible
- 3 L'Agent vérifie que le processus existe toujours (os.kill(pid, 0))
- 4 L'Agent exécute kill -9 <PID> via subprocess

Cas d'utilisation 6 : Arrêter automatiquement un processus malveillant (suite)

Scénario nominal (suite)

- 5 Le signal SIGKILL est envoyé au processus
- 6 Le noyau termine immédiatement le processus
- 7 L'Agent enregistre l'action en base et dans le fichier de log

Scénario alternatif / Exceptions

⚠ Processus déjà terminé → action ignorée, log enregistré avec statut 'déjà arrêté'

⚠ PID réutilisé (race condition) → vérification du nom de processus avant kill

⚠ Droits insuffisants → alerte système, escalade à l'administrateur

Cas d'utilisation 7 : Consulter le dashboard de supervision

III — Cas d'utilisation

Intitulé

Consulter le dashboard de supervision

Objectif

Permettre à l'administrateur d'accéder à l'interface web du HIPS pour visualiser l'état du système, les alertes en cours et les indicateurs de sécurité en temps réel.

Résumé

L'administrateur accède au dashboard via un navigateur web. L'interface affiche en temps réel les alertes actives, le nombre d'événements, les IP bloquées et l'état du service HIPS avec un code couleur de niveau de menace.

Acteurs

Internes : Administrateur système

Externes :

Préconditions

- L'administrateur est authentifié sur le dashboard
- Le service HIPS est en cours d'exécution
- Le serveur web Flask/Django est accessible depuis le navigateur

Postconditions

- L'administrateur visualise l'état de sécurité actuel
- Les alertes sont marquées comme 'vues' dans la base de données
- Le tableau de bord est actualisé en temps réel

Scénario nominal

- 1 L'administrateur ouvre son navigateur et accède à l'URL du dashboard
- 2 L'interface web charge la page d'accueil
- 3 Le dashboard envoie une requête au backend
- 4 Le backend interroge la base de données pour les alertes récentes

Cas d'utilisation 7 : Consulter le dashboard de supervision (suite)

Scénario nominal (suite)

- 5 Les données sont retournées et affichées en temps réel
- 6 L'administrateur visualise les indicateurs (alertes, IP bloquées, état HIPS)

Scénario alternatif / Exceptions



Session expirée → redirection vers la page de login



Backend indisponible → message d'erreur de connexion affiché



Aucune alerte → tableau de bord vert 'Système sécurisé'

Cas d'utilisation 8 : Consulter l'historique des alertes

III — Cas d'utilisation

Intitulé

Consulter l'historique des alertes

Objectif

Permettre à l'administrateur de consulter, filtrer et exporter l'historique complet des événements et alertes enregistrés par le système HIPS.

Résumé

Via le module Historique du dashboard, l'administrateur peut naviguer dans tous les événements passés, appliquer des filtres (date, type, sévérité) et exporter les résultats au format CSV pour audit ou archivage.

Acteurs

Internes : Administrateur système, Agent HIPS

Externes :

Préconditions

- L'administrateur est authentifié
- Des événements ont été enregistrés en base de données
- Le module Historique est disponible dans le dashboard

Postconditions

- L'administrateur a accès aux événements filtrés
- Un fichier CSV peut être généré et téléchargé
- Les événements consultés peuvent être marqués comme traités

Scénario nominal

- 1 L'administrateur accède au module Historique via le menu
- 2 Il sélectionne les filtres souhaités (période, type, sévérité)
- 3 Le frontend envoie la requête au backend avec les paramètres
- 4 Le backend exécute la requête SQL filtrée

Cas d'utilisation 8 : Consulter l'historique des alertes (suite)

Scénario nominal (suite)

- 5 Les résultats sont retournés et affichés sous forme de tableau
- 6 L'administrateur peut cliquer sur un événement pour voir le détail
- 7 L'administrateur peut exporter les résultats en CSV

Scénario alternatif / Exceptions

! Aucun résultat pour les filtres → message 'Aucun événement trouvé'

! Trop de résultats → pagination automatique (100 entrées par page)

! Erreur d'export → notification d'erreur, réessai possible

Intitulé

S'authentifier sur le dashboard

Objectif

Permettre à l'administrateur de se connecter de manière sécurisée au dashboard web du HIPS en vérifiant son identité via un formulaire de login.

Résumé

L'administrateur saisit ses identifiants sur la page de connexion. Le système vérifie le mot de passe hashé en base de données (bcrypt). En cas de succès, une session sécurisée est créée et l'administrateur est redirigé vers le dashboard.

Acteurs

Internes : Administrateur système

Externes :

Préconditions

- Le serveur web HIPS est en cours d'exécution
- Le compte administrateur est créé en base de données
- L'administrateur dispose de ses identifiants valides

Postconditions

- Une session sécurisée est créée pour l'administrateur
- L'administrateur est redirigé vers le dashboard principal
- La tentative de connexion est journalisée en base

Scénario nominal

- 1 L'administrateur accède à l'URL du dashboard
- 2 Le serveur retourne la page de login (non authentifié)
- 3 L'administrateur saisit son identifiant et son mot de passe
- 4 Le formulaire est soumis via POST HTTPS

Scénario nominal (suite)

- 5 Le backend récupère le hash bcrypt du mot de passe en base
- 6 Le hash est comparé avec le mot de passe saisi
- 7 Correspondance OK → session créée, redirection vers le dashboard

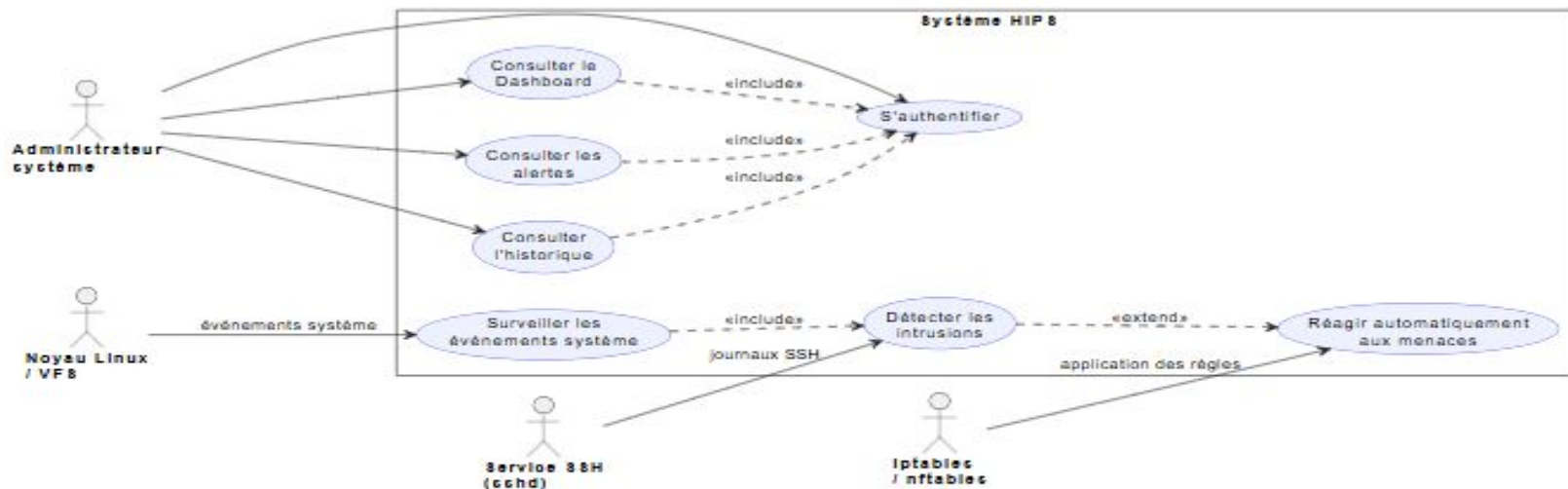
Scénario alternatif / Exceptions

⚠ Identifiants incorrects → message d'erreur, compteur d'échecs incrémenté

⚠ 3 tentatives échouées → blocage temporaire du compte (30 minutes)

⚠ Session expirée → déconnexion automatique, retour à la page de login

D. Diagramme de cas d'utilisation



CONCLUSION

Synthèse de l'analyse et perspectives

Dossier d'analyse – Système HIPS

Merci pour votre attention

Questions ?

KAMDOUM TEUFENG Josette

HARVIS LANDRY FOTSEU FONKWA

YAKAM TCHAMOU Rick Vadel

Encadrant : NGUIMBUS Emmanuel